

AnthBlack Systems Rebuild Manual

Controlled reconstruction of Kaiku, Provenance Core, Echo, Audio Relay, research, education, and supporting systems

Author: Dr. William James Stanley

Version 0.1.0 | Reconstruction candidate | 11 September 2026

This manual distinguishes present evidence from historical reports and proposals. It does not authorize destructive cleanup, canonical-path changes, governance changes, publication, or release without the Human Gate.

Source archive SHA-256: 21387100f8fb1594f0ecce32082bb81fec0d2cb26041c417e6d0b621853572e8

Exact Markdown SHA-256: 0510b860982ba673d0f6e1da9a46fe9f583b5fc39ba4788c9da7de6c45b70731

AnthBlack Systems Rebuild Manual

Purpose and controlling conclusion

This manual is a reconstruction map, not proof that every described artifact still exists. Its purpose is to let Dr. William James Stanley recover the system without again rebuilding the whole conceptual universe from memory.

The controlling conclusion is simple: rebuild the evidence base first, then the common substrate, then one project at a time. Within each project, inspect one boundary, change one boundary, test it, preserve the result, and stop at the first failure. Nothing in this manual authorizes blanket synthesis, speculative refactoring, destructive cleanup, or silent replacement of missing artifacts.

The source archive is a composite conversation record. It contains repetition, historical and current variants, reports of generated packages, proposed architectures, user decisions, and assistant claims. Repetition does not increase evidential weight. A reported test is historical evidence of a reported test, not proof that the presently installed copy passes. A filename is evidence that an artifact was referenced, not proof that it remains available.

How to use this manual

Use this document in three passes.

1. Inventory: establish what physically exists now without modifying it.
2. Reconcile: map present artifacts to the states in this manual and mark conflicts.
3. Rebuild: proceed through the gates in order, preserving evidence after every material step.

Never execute a later phase merely because an earlier phase is described here. Each phase begins only after its entry conditions are demonstrated on the current machine.

Epistemic labels

Label	Meaning
[USER]	Direct user instruction, decision, description, or user-originated concept in the supplied evidence
[OBS]	Direct observation recorded in supplied terminal output, file inspection, or current workspace inspection
[HIST-REPORT]	A historical report that something was built, tested, installed, or observed; requires present re-verification
[AI-P]	Assistant proposal or interpretation, not a user decision
[IF]	Inference from available evidence
[UN]	Missing, conflicting, or not yet verified
[FAL]	Falsified by stronger evidence
[SUPERSEDED]	Historical state retained for lineage but not current authority

State vocabulary

The following states must not be collapsed: IDEA, PROPOSED, DESIGNED, CANDIDATE, IMPLEMENTED, INSTALLED, RUNNING, TESTED, VERIFIED, ACCEPTED, FROZEN, and SUPERSEDED.

Examples:

- A ZIP referenced in a chat is not necessarily present.

- Source code present on disk is not necessarily installed.
- Installed software is not necessarily running.
- A test report is not verification of a different copy.
- One successful launch does not verify an architecture.
- A model response stored in provenance is preserved output, not accepted fact.

Part I — Governance of the rebuild

1. Operating cycle

The controlling cycle is:

observe → distinguish → verify → act minimally → test → preserve → update state

The central meta-rule is:

Never substitute the system's representation for the system itself.

Operational consequences:

- Current direct user statements outrank historical interpretations.
- Current machine and file observations outrank remembered architecture.
- Attached source material must be inspected before system-level conclusions are made.
- Missing information stays missing until observed.
- Failures are preserved when they explain the present state.
- Derived state should be disposable whenever an authoritative source exists beneath it.
- No silent fallback is permitted when an authoritative path or source is unavailable.

2. Authority boundary

Human authorization is required before:

- deleting, moving, overwriting, or renaming canonical material;
- replacing a current installation with a candidate;
- changing locked Kaiku rules;
- changing authoritative data locations;
- publishing, sending, or sharing work externally;
- enabling consequential tools or write authority;
- declaring a module accepted, frozen, or released.

The AI may inspect, compare, propose, test in a bounded copy, and produce a candidate record. It may not convert its proposal into a user decision.

3. Private-source boundary

The default evidence boundary is the current conversation plus explicitly supplied material. Historical email, Drive, Calendar, private repositories, connected services, and old uploads must not be searched merely to complete the picture. If a particular private source becomes necessary, identify the exact source, desired object, and reason, then obtain narrow permission.

4. Universal module gate

Every software module uses the same gate:

1. Identify the exact file and hash it.
2. Read it in full.
3. State its single responsibility.
4. State inputs, outputs, persistent effects, dependencies, and forbidden responsibilities.
5. Trace the normal path forward from input to output.
6. Trace backward from output to every required precursor.
7. Run existing tests without modification.
8. Preserve the baseline results.
9. Make at most one consequential change.
10. Re-run the narrow test, then the module suite.
11. Record the change, reason, result, and new hash.
12. Stop on failure. Do not compensate downstream.
13. Temporarily freeze the module before proceeding.

5. Recovery pattern

For each authoritative artifact:

discover → identify → hash → copy → verify copy → operate on copy → test → compare → obtain
 Human Gate → promote → preserve predecessor

Do not delete a predecessor merely because a successor exists. Mark supersession explicitly.

Part II — Project topology and boundaries

6. Umbrella and project map

AnthBlack Systems is the umbrella identity. The projects beneath it remain distinguishable.

Project	Responsibility	Current evidential state	Must remain separate from
Provenance Core	Authoritative event and lineage substrate	Historical implementation reports; current installed copy must be inspected	Context indexes, model prose, GUI
Kaiku	Persistent local personal/cognitive system	Lean V6 and Personal 0.2 variants reported; canonical current copy unresolved	Echo, Audio Relay, business/coursework
Echo	Terminal, diagnostic, provenance and control surface	Windows PowerShell ancestor documented; Linux form unresolved	Kaiku product identity
Audio Relay	Replaceable speech transport and voice adapters	Linux package installation partly reported; end-to-end state unresolved	Kaiku core authority
Observation Deck	Read-only rendered-pixel observation system	Conceptual/parked	Write/control path
Kognitiv Akademia	Education/tutoring business	Offer and positioning defined; operating assets incomplete	Higher Research
Higher Research	Research-platform family	Components proposed; boundaries not fully frozen	Kognitiv tutoring business

Project	Responsibility	Current evidential state	Must remain separate from
Kirtouma	Language/governance language project	Two incompatible definitions recorded; redesign required	Other projects until resolved
Chat Manifest / Archive Recovery	Recover conversations and artifact lineage	Historical parser/evidence package reported	Kaiku memory itself
University and research work	Course and manuscript work	Several active/historical items	Product code repositories
OS Commissioning	Reproducible Linux machine substrate	Mint commissioning partly observed/reported	Individual app code

7. Physical hierarchy target

The user previously directed a single laboratory root at:

`/home/anthraxblack/Desktop/Anthrax_Blacks_Laboratory`

This path is a historical/current user-directed target, not verified in this workspace. Before creating or moving anything on the user's machine, inspect the current tree and obtain confirmation of the final numbered layout.

Recommended structural intent, without asserting exact numbering:

- foundation and governance;
- Provenance Core;
- Kaiku;
- Echo;
- Audio Relay;
- Observation Deck;
- Higher Research;
- Kirtouma;
- Kognitiv Akademia;
- University subjects;
- archives and superseded builds.

Do not create one repository named everything. Shared standards may be referenced without erasing project boundaries.

Part III — Foundation rebuild

8. Phase 0 current-state capture

Entry condition

Access to the target Linux machine with no intent to modify it during this phase.

Capture

- operating system release and kernel;
- user identity and home path;
- mounted filesystems and free space;

- full laboratory tree with permissions and timestamps;
- Git repositories, branches, remotes, and dirty state;
- Python interpreters and virtual environments;
- Ollama installation, service state, and model inventory;
- systemd user/system services relevant to projects;
- installed desktop launchers and autostart entries;
- Audio Relay paths, services, and hotkey bindings;
- hashes of all candidate manifests, ZIPs, ledgers, databases, configuration, and source files;
- trash and obvious recovery locations, read-only;
- external drives, if attached and explicitly in scope.

Outputs

- timestamped inventory text;
- sorted file manifest with SHA-256 where practical;
- environment report;
- conflict list;
- zero changes.

Stop conditions

Stop if the expected home, root filesystem, or laboratory path differs from the current user statement; if a filesystem is degraded; or if inventory suggests active writes to material being recovered.

9. Phase 1 artifact registry

Create an append-only registry containing:

- exact path;
- filename;
- size;
- modification time as observed;
- SHA-256;
- project;
- origin if known;
- lifecycle state;
- canonical status;
- predecessor/successor relationship;
- supporting source;
- uncertainty.

Do not deduplicate solely by filename. Identical names with different hashes are distinct candidates. Identical hashes at different paths are duplicate byte representations, not automatically disposable copies.

10. Phase 2 canonical selection

For each project, compare candidates using:

1. current direct user designation;
2. current observed working installation;
3. manifest/hash relationships;
4. test evidence tied to the exact bytes;
5. timestamps and historical descriptions;
6. completeness;
7. inference only when clearly labelled.

If no candidate can be established as canonical, retain all and mark UNRESOLVED. Never let a cleaner-looking package silently win.

Part IV — Provenance Core rebuild

11. Responsibility

Provenance Core preserves what occurred, who or what originated it, the event sequence, transformation links, and integrity evidence. It does not establish truth, factual correctness, originality, inventorship, or completeness.

12. Historical implementation claims requiring re-verification

The archive reports a Module 1 implementation with:

- canonical UTF-8 JSONL;
- SHA-256 event hashes;
- prev_hash chaining;
- event UUIDs;
- separation of origin and kind;
- validation before write;
- fail-closed append;
- strict replay;
- POSIX writer locking;
- fsync durability intent;
- narrow recovery of a trailing partial record;
- refusal to repair internal corruption automatically;
- sensitive-file permission handling;
- nine reported tests passing.

These are [HIST-REPORT] until tied to the exact current file hashes and rerun.

13. Known unresolved defects and hardening questions

The archive records one concrete permission defect in one candidate: recovery backup files were reportedly created under normal umask and changed to 0600 only after data had been copied. The required property is creation as 0600 from the first filesystem operation.

Additional questions reported but not accepted as mandatory changes:

- Is `event_id` structurally validated as a UUID?
- Is `schema_version == 1` enforced?
- Is timestamp syntax validated?
- Can a valid suffix be deleted without external anchoring being able to detect it?

The last limitation cannot be solved by an internally self-contained hash chain alone.

14. Provenance rebuild sequence

1. Locate every `provenance.py`, `test`, `manifest`, `verification` report, and ledger candidate.
2. Hash and register all candidates.
3. Select one baseline only after byte-linked evidence comparison.
4. Read the module and tests in full.
5. Compile without modifying.
6. Run tests in an isolated temporary directory.
7. Inspect created permissions at creation time, not merely final permissions.
8. Test tamper detection, partial tail, internal corruption, blank-line damage, duplicate IDs, stale writers, and concurrent writers.
9. Confirm replay verifies before exposing events.
10. Confirm append validates and verifies existing history before allocation/write.
11. Confirm failure does not mutate the ledger.
12. Preserve test output and hashes.
13. Repair one defect only if observed in the selected current candidate.
14. Re-run all Module 1 tests.
15. Human Gate: ACCEPT, REVISE, REJECT, DEFER, or VERIFY.

Exit criteria

- exact module and test hashes recorded;
- all required tests pass on the current machine;
- new sensitive files are 0600 at creation;
- malformed history fails closed;
- recovery is narrow, explicit, and preserves the damaged original;
- no dependent module was changed.

Part V — Kaiku rebuild

15. System identity

Kaiku is a persistent local-first personal/cognitive system. The language model is a replaceable component. Kaiku owns continuity, provenance, governed context, user adaptation, recovery, and action boundaries outside the model.

The locked distinction is:

Qwen is not Kaiku. Mistral is not Kaiku. The model is a component of Kaiku.

16. Lean module topology

Module	Single responsibility	Authoritative or derived
rules.py	Canonical locked rules and runtime normalization	Static governed configuration
provenance.py	Append, verify, replay, and narrow recovery	Authoritative persistent evidence
context.py	Rebuildable searchable context index	Derived persistent representation
model.py	Narrow local Ollama interface	Replaceable runtime adapter
core.py	One-turn orchestration and causal ordering	Coordinator
kaiku.py	GUI and human interaction	Disposable presentation
config.json	Runtime settings	Configuration
interaction_profile.json	General interaction behaviour	Versioned static profile, not biography
future user-state module	Revisable person-specific interpretation	Derived persistent representation

17. Required normal turn story

The desired causal order is:

1. User submits a message.
2. The user message is durably recorded with `origin = USER`.
3. The context index is updated from that preserved event.
4. Relevant recent and older records are selected.
5. The exact working context is constructed and identified.
6. The local model is called.
7. Failure or output is durably recorded with correct origin.
8. Derived indexes are updated.
9. Only then is the durable response displayed as completed output.

For real streaming, distinguish provisional display chunks from the final durable model message. Do not pretend streamed pixels are already authoritative state. Define how interrupted streams are preserved before enabling streaming.

18. Startup and shutdown story

The preferred causal order is:

process launch → imports define capabilities → core constructed → configuration loaded → locked rules/profile verified → provenance opened and verified → derived index reconciled → model adapter constructed → APP_START recorded → SESSION_START recorded → GUI waits

The archive notes that at least one candidate reportedly recorded SESSION_START before APP_START. Inspect the current code; repair only if the defect is present and after its owning module reaches the gate.

Shutdown should record APP_STOP when safe, close databases and threads cleanly, and leave authoritative state valid if the model or GUI fails.

19. Rules and philosophical structure

The original source rules are preserved as historical/canonical text. Runtime normalization may remove duplicates but must not silently change meaning.

The governing distinctions include:

- representation is not reality;
- reasoning is not evidence;
- meaning is not fact;
- the model is a component, not the system;
- claims are constrained by evidence;
- uncertainty remains visible;
- provenance is preserved;
- governance is exposed;
- material failure is preserved;
- recovery is designed;
- categories and boundaries follow evidence rather than convenience.

Origin, claim type, and evidence status remain orthogonal:

Dimension	Example values
Origin	SYSTEM, USER, MODEL, TOOL, SOURCE
Claim type	OBSERVATION, CALCULATION, INFERENCE, ASSUMPTION, PROPOSAL
Evidence status	UNASSESSED, SUPPORTED, CONTRADICTED, INSUFFICIENT

A complete message is not automatically one claim. Falsifiability checks apply to empirical hypotheses and predictions where meaningful, not every sentence.

20. Components explicitly excluded from the lean baseline

Do not restore these merely because they appeared in earlier candidates:

- REFLEX and EXECUTIVE graphs;
- pseudo-AIF machinery;
- unused co-occurrence graphs and edge weights;
- a decorative `PolicyKernel` without a real action boundary;
- brain/cognitive metaphors used as architecture;
- uncalibrated probability-like belief vectors;
- ontology platforms without an observed need;
- explicit conversational mode theatre.

Plain names are preferred: input annotation, response audit, context index, term, authorization policy, operator approval.

21. Context and retrieval

Provenance is authoritative. `context.sqlite3` is derived and must be rebuildable from provenance.

The archive describes a candidate using:

- sessions, messages, terms, and a ledger synchronization marker;
- recent dialogue, historically around eight messages;
- related older messages, historically around four;
- lexical term overlap weighted by rarity;
- a bounded search over roughly 1,500 candidate messages;
- deduplication between recent and related sets;
- truncated historical references.

These are candidate implementation details, not eternal rules.

Known limitations:

- lexical retrieval misses semantic equivalents;
- preservation does not guarantee retrieval;
- a 1,500-message candidate bound can hide older relevant evidence;
- storing model output and retrieving it repeatedly can create a model-generated feedback loop;
- a context hash without preserving exact rules/profile versions may not permit exact prompt reconstruction.

Required tests:

- rebuild index from a known ledger and compare deterministic state;
- delete/corrupt derived state and recover without altering provenance;
- same-session retrieval;
- recent selection order;
- duplicate exclusion;
- origin-aware retrieval;
- explicit exclusion or down-weighting policy for unaccepted model interpretations;
- trace every selected item back to its provenance sequence/hash;
- prove the current user message appears exactly once in the packet.

22. Model adapter

The model adapter must do only the following:

- accept a prepared message packet and runtime options;
- enforce the allowed local endpoint boundary;
- call Ollama;
- expose streamed or non-streamed outputs according to an explicit contract;
- return model identity and available runtime measurements;
- preserve errors without inventing success.

Historical model states conflict:

- Qwen3:4B is repeatedly named as the configured/target model.
- One inspected variant reportedly defaulted to `mistral:latest`.
- The user's later preference is to prioritize Mistral-family models when their actual conversation quality is better, while retaining Qwen as comparison/fallback.

Therefore the model is UNRESOLVED until current config.json, ollama list, and acceptance tests are inspected. Do not settle it from specifications alone.

Required adapter tests:

- reject non-loopback model URLs;
- confirm exact configured model exists;
- record model identity per response;
- timeout and unavailable-service paths;
- malformed response path;
- interrupted streaming path if enabled;
- no database or user-profile access inside the adapter.

23. Core orchestration

Inspect core.py only after provenance, context, model, and rules/profile boundaries are temporarily frozen.

Forward audit:

input → durable user event → index → selection → compiled context → context record → model call
→ durable failure/output → index → display result

Backward audit from a displayed answer:

displayed answer → durable model event → model identity and context hash → compiled packet →
selected message IDs/hashes → source events → valid ledger chain

Any displayed completed answer without that backward path is a defect.

24. GUI

The accepted direction is a clean, predominantly white thinking surface:

- broad open conversation canvas;
- thin left rail;
- restrained black typography;
- pale blue/lavender structural curves;
- minimal custom title area;
- narrow pill composer;
- paperclip and circular send control;
- conversations, memory, why/history, and settings kept peripheral;
- machinery does not dominate the canvas.

Avoid dashboard cards, dark enterprise styling, excessive chat bubbles, glassmorphism, heavy borders, and control panels for model parameters.

The GUI is not memory. It must not contain provenance algorithms, retrieval logic, or direct Ollama orchestration. A GUI preview and fake-model test are not a live-model acceptance test.

25. Historical Kaiku artifacts to locate

The source archive references, without proving present availability:

- kaiku_v6_lean_candidate_20260908.zip;

- Kaiku Personal 0.2 package, historically reported SHA-256
4b836c497e56eff2cf09054e06c1302cf7db42f29aab129471d91937abe98331;
- guided candidate ZIP, historically reported SHA-256
f1c67acfb9366d9306ca802329b60b6781dd17336dc31294f1fc56f7fc2351f0;
- ARTIFACT_MANIFEST.json;
- MODULE1_VERIFY.md and PDF;
- TEST_OUTPUT.txt;
- SHA256SUMS.txt;
- test_provenance.py;
- README.md, VERIFY.md, VERIFY_REPORT.md;
- GUI_PREVIEW.png and DESIGN_REFERENCE.png;
- learning checkpoints, teaching prompt, and Brett manifest artifacts.

Treat a matching hash as strong byte-identity evidence. Treat a matching filename alone as weak evidence.

26. Kaiku sequential rebuild order

1. Provenance Core.
2. Context index.
3. Model adapter.
4. Locked rules and interaction profile.
5. Core orchestration.
6. GUI.
7. Blank user-learning layer.
8. Critical/philosophical interaction tests.
9. One bounded action/tool capability.
10. Streaming.
11. Speech integration through Audio Relay.
12. Packaging and Brett acceptance test.

If a later layer exposes an earlier defect, return explicitly to the owning module and repeat its gate. Do not silently patch across layers.

Part VI — Blank personal learning and Brett acceptance build

27. Design objective

The same Kaiku software starts blank about each person and becomes specific through preserved interaction. Do not hard-code Brett's PhD field, mathematical level, interests, personality, loneliness, or preferences. Respond to demonstrated competence in the current domain, not biography or global intelligence labels.

28. Evidence layers

Keep at least two conceptual layers:

1. Episodes: what was actually said, done, corrected, accepted, or rejected.
2. Derived personal state: the current revisable interpretation of those episodes.

The derived layer must be disposable and rebuildable. Explicit correction outranks inference. Old or conflicted preferences become inactive or superseded rather than deleted.

Suggested lifecycle:

single indication → candidate → repeated support → supported → explicit confirmation → confirmed

Conflict produces CONFLICTED; explicit correction produces SUPERSEDED; context-dependent or outdated material may become INACTIVE.

29. Feedback-loop protection

The system must distinguish:

- a user's direct statement;
- a model inference about the user;
- a later retrieval of that model inference;
- fresh user confirmation or correction.

Repeated retrieval of a model's own assumption must never count as repeated user evidence.

30. Interaction profile

General interaction behaviour may include:

- conversational and direct;
- challenge invalid reasoning without a praise economy;
- no patronizing explanations;
- infer working depth from demonstrated competence;
- tolerate unfinished ideas and incomplete mathematics;
- keep uncertainty explicit;
- explain differently when an explanation fails;
- become terse and operational when asked to fix something;
- allow teaching depth to emerge from the task rather than an announced mode.

Teaching sequence when appropriate:

concept → mechanism → prediction → small exercise or command → expected observation → actual evidence → interpretation

31. Brett acceptance criteria

The first living build aims for:

- clean Kaiku GUI;
- natural local conversation;
- blank initial personal state;
- continuity across restart;
- inspectable and correctable derived memory;
- locked rules verified by hash;

- critical/philosophical engagement;
- preserved unresolved arguments;
- one genuine bounded capability;
- provenance underneath without making provenance the user's hobby;
- fun sufficient that Brett chooses to continue using it.

The acceptance session should not be over-scripted. The system should be given to Brett with minimal choreography so his self-directed uses become evidence. Architecture approval by Brett is not the primary success measure; voluntary reuse is.

Part VII — Echo rebuild

32. Identity and role

Echo is the terminal, diagnostic, and control surface. It is not the name of Kaiku. Historical references that use Echo as the whole conversational product are superseded unless the user changes the decision.

Echo should expose:

- provenance status;
- project/session identity;
- changes and diffs;
- missing relations and broken lineage;
- unresolved contradictions;
- failed or abandoned operations;
- derived state that no longer matches its source;
- explicit gaps in recording;
- Human Gate state;
- recovery and verification controls.

It must not claim literal Hodge-theoretic implementation merely because residuals, gaps, and relational structure inspired the design.

33. Windows ancestor

Historical artifacts referenced include:

- 20260809_ANTHBLACKSYSTEMS_TERMINAL_PROVENANCE_V1_4_README.md;
- 20260809_D_KAIKU_FOLDER_TREE_V1_4.txt;
- 20260809_PACKAGE_MANIFEST_SHA256.txt;
- 20260809_STATIC_SANITY_CHECK.json;
- terminal provenance profile, installer, rollback, and verification PowerShell scripts;
- 20260822_INSTALL_KAIKU_TERMINAL_ARCH_V1_5_PROFILE_V1_4_FINAL.ps1;
- 20260809_TERMINAL_CONFIG_V1_4.json.

The Windows implementation is historical evidence, not Linux-ready code. Recover its requirements and tests before porting behavior.

34. Echo rebuild order

1. Locate and hash historical package artifacts.
2. Extract behavior requirements from README, scripts, and tests.
3. Separate portable semantics from PowerShell/Windows mechanics.
4. Define the Linux command surface without changing Kaiku.
5. Implement session identity and read-only status first.
6. Implement provenance verify and context consistency reports.
7. Implement explicit Human Gate-controlled actions one at a time.
8. Implement recording-gap and unclean-session recovery behavior.
9. Restore restrained `<>` `[]` `<>` `[]` visual identity last.
10. Test rollback for every write-capable function.

Part VIII — Audio Relay and cross-device observation

35. Audio Relay boundary

Audio Relay is a stable speech transport around replaceable adapters:

- microphone capture;
- Whisper/faster-whisper transcription;
- Piper or Sherpa speech synthesis;
- hotkey and accessibility adapters;
- laptop and phone endpoints.

It does not own Kaiku's memory, governance, or write authority.

36. Reported Linux state requiring inspection

The current conversation context reports:

- package `Audio_Relay_Clean_20260910_v3`;
- `INSTALL_EVERYTHING.sh`;
- successful package and virtual-environment dependency installation;
- faster-whisper 1.2.1, ctranslate2 4.8.2, piper-tts 1.8.0, sounddevice 0.5.5, and pynput 1.8.2;
- an earlier failure involving `~/.local/share/kaiku/audio-relay/.venv`;
- a later intended path of `~/.local/share/audio-relay`;
- desired hotkeys approximately `Alt+Shift+O` record, `Alt+Shift+P` transcribe, `Alt+Shift+I` read back, `Alt+Shift+X` exit;
- Pixel 10 with Sherpa TTS and TTS Util available;
- end-to-end live behavior not yet established here.

Because two install roots are recorded, the current authoritative path is UNRESOLVED until inspected. No silent fallback between them.

37. Audio Relay rebuild order

1. Inventory both historical paths, launchers, services, and hotkey definitions.

2. Hash package and installed code.
3. Establish which installation, if any, is current.
4. Test microphone enumeration only.
5. Test a fixed-duration local recording.
6. Test transcription from a known audio fixture.
7. Test synthesis from a known text fixture.
8. Test playback.
9. Test each hotkey independently for collisions.
10. Test the complete record → transcribe path.
11. Test text → speech → stop path.
12. Integrate with Kaiku only after standalone acceptance.
13. Preserve identical behavior contracts across PC and phone where platform capabilities allow.

38. Observation Deck and pixel language

The parked cross-platform concept treats rendered pixels as a universal observation representation. Different applications and operating systems expose state visually; a read-only observer may reconstruct rendered text and structure from that channel.

Critical separation:

visual/pixel observation is read-only; write/control authority is separate, explicit, local, and human-governed.

Rebuild sequence when resumed:

1. Capture known screenshots/video without control authority.
2. Recover text regions and timing.
3. Reconstruct temporal overlap into a stable rendered-text stream.
4. Parse structural units.
5. Compare reconstruction against known source.
6. Measure drift, omission, and duplication.
7. Keep any keyboard/accessibility/write adapter in a separate process and authorization boundary.

Part IX — Archive recovery and working-history system

39. Chat Manifest responsibility

This project converts exported or rendered conversations into recoverable artifacts and registries. It is motivated by missing chats and repeated reconstruction work. It is not Kaiku's live context database.

Historically reported outputs include:

- one PDF per conversation;
- an Excel registry;
- a JSONL hash manifest;
- logs;

- ReportLab and OpenPyXL dependencies;
- a parser series including a v03 truncation fix.

All must be located and rerun against fixtures before reliance.

40. Working-history lifecycle

For material conversations:

1. Identify an existing canonical Markdown history if one exists.
2. Read it before relying on state.
3. Update it only for material events.
4. Record files, decisions, tests, failures, canonical changes, and unresolved issues.
5. Preserve supersession instead of silently rewriting history.
6. At closeout, reread the complete conversation and compare against the running history.
7. Produce the required Markdown and PDF derivatives with source binding.
8. Leave a continuation state.

The running history is a compact reconstruction record, not a transcript.

Part X — Kognitiv Akademia rebuild

41. Purpose and positioning

Kognitiv Akademia is the education business. Its core positioning is:

Understand the biology. Read the evidence. Think like a scientist.

Primary audience:

- university biomedical students;
- Honours, Masters, and early PhD researchers;
- VCE Biology later, after content/audience audit.

Historical test pricing:

- university tutoring: A\$70/hour;
- research mentoring: A\$80/hour;
- writing coaching: A\$80/hour;
- VCE later: A\$65/hour;
- free 15-minute fit call.

These are candidate market prices, not immutable rules.

42. Teaching method

The offer is scientist-led understanding, not generic AI tutoring. The teaching loop is:

diagnose misconception → explain mechanism → test understanding → learner explains back → transfer to a new case → withdraw scaffolding

Assessment-integrity check:

Could the learner explain and defend the work orally?

43. Rebuild sequence

1. Recover the current offer sheet, pricing decisions, bios, service-page copy, outreach drafts, posters, and design references.
2. Register canonical/current versions and superseded drafts.
3. Confirm which channels are live: StudentVIP, LinkedIn Service Page, referrals, student societies, and later Learnmate.
4. Confirm current contact details before publication.
5. Reconstruct one concise service page.
6. Reconstruct one university biomedical offer.
7. Reconstruct one early-researcher offer.
8. Reconstruct outreach for Monash societies using the existing angles.
9. Build one diagnostic first-session protocol.
10. Build one mechanism-based sample lesson.
11. Test messaging with real enquiries before expanding assets.
12. Preserve outcomes and revise pricing/positioning from evidence.

Design direction currently recorded: functional and striking, light beige/white, maroon hints, old typewriter character, graphic lines, no black-heavy poster treatment, and no generic corporate aesthetic.

Part XI — Higher Research and governed reasoning research

44. Boundary

Higher Research is a separate research-platform family. Historically proposed components include Statistics Synthesiser, evidence/reference systems, DocSynth, research administration, and governed research workflows. Do not automatically merge it with Kognitiv Akademia.

45. Core empirical proposition

The research proposition is provisional:

Explicit provenance, epistemic classification, governed context construction, verification, and human authority around an otherwise unchanged language model may improve traceability and epistemic reliability.

The word may is essential. This is an empirical hypothesis, not a project slogan treated as fact.

46. Controlled essay experiment

Potential conditions:

- A — Baseline: ordinary LLM with same sources and task.
- B — Output governance: same model, with post-generation claim decomposition and checking.
- C — Full Kaiku: provenance ingest, atomic observations, classification, governed context, LLM generation, claim decomposition, verification, Human Gate, final rendering.

Controls:

- same model/version;

- same corpus and question;
- same tool access;
- comparable context/output limits;
- exact input and raw output preservation;
- predefined hypotheses and metrics;
- repeated stochastic runs;
- blinded evaluation where possible;
- no silent post hoc repair.

Metrics:

- unsupported claims;
- recoverable support links;
- contradiction preservation;
- escalation from association to causation or hypothesis to fact;
- substantive human correction burden;
- process overhead.

If governance adds bureaucracy without improving reliability, preserve that as a failure.

Part XII — Kirtouma rebuild

47. Current conflict

Kirtouma is an established language-related project name. Two incompatible definitions are recorded:

- a constraint-solving or structured language concept;
- a governance kill-switch/control language concept.

The project is therefore REDESIGN REQUIRED. Do not implement until the object boundary is restored.

48. Resolution protocol

1. Recover all Kirtouma sources and repository state.
2. Create a claim/definition table with source provenance.
3. Separate shared syntax ideas from distinct responsibilities.
4. Determine whether the two definitions are versions, layers, or separate projects.
5. Human Gate chooses the retained boundary and name(s).
6. Only then write a grammar, interpreter, threat model, or integration plan.

Part XIII — University and manuscript work

49. PSY4081 Perception and Cognition

Current course topics include Gestalt organization, bottom-up and top-down processing, predictive coding, Bayesian inference, and perceptual illusions. Assignment 1 was described as an article-based literature review/essay with a high-grade target.

Rebuild protocol:

1. Recover the official task sheet, rubric, chosen article, course readings, notes, and due date from currently supplied/local materials.
2. Treat official instructions as authoritative over remembered descriptions.
3. Build an evidence table before prose.
4. Separate article claims, external evidence, inference, and critique.
5. Use the work as a governed-reasoning experiment only if doing so does not endanger the assessment deadline or integrity rules.
6. Preserve drafts and feedback longitudinally.

50. PSY4062 manuscript

Historical analysis reported:

- emerging adulthood/Feeling In-Between and perceived stress predicting life satisfaction;
- total N approximately 2,004;
- complete-case N approximately 1,999;
- R^2 approximately .252;
- stress beta approximately -.498;
- Feeling In-Between beta approximately +.081.

These are historical summary values. Rebuild from the canonical dataset, script, output, and manuscript before reuse or publication.

51. Cell death review concept

Crossing the Point of No Return concerns regulated cell death as a state-space/commitment-transition problem. Current status is unresolved. Recover notes and provenance before expanding the theory.

Part XIV — OS commissioning and reproducibility

52. Layer order

The intended machine hierarchy is:

clean Linux machine → AnthBlack commissioning → Provenance Core → Kaiku → model runtime → optional modules

Recorded machine facts from current context include Linux Mint 22.3, kernel 6.14.0-37, ext4 root on nvme0n1p2, EFI on nvme0n1p1, LightDM, Cinnamon/X11, and an approximately 40.5-second boot. These require current re-observation before being treated as present state.

53. Commissioning rebuild sequence

1. Read-only identity, disk, mount, boot, service, package, and session inventory.
2. Preserve a baseline manifest and checksums.
3. Recover boot-chain and Mint audit history.
4. Confirm recovery media and rollback paths.
5. Apply one configuration class at a time.
6. Verify after each class.
7. Install Provenance Core before application modules that depend on it.

8. Install Kaiku and model runtime.
9. Add Audio Relay and other modules only after standalone tests.
10. Produce a reproducible bootstrap script only after the manual sequence is proven.

The bootstrap script is the compiled result of a verified commissioning procedure, not a substitute for discovering the procedure.

Part XV — Verification, packaging, and handoff

54. Project acceptance matrix

Project	Minimal acceptance evidence
Provenance Core	Current exact tests pass; fail-closed; recovery preserves damaged source; correct permissions at creation
Kaiku baseline	Local model conversation; restart continuity; valid provenance; rebuildable context; backward trace from displayed answer
User learning	Derived state rebuilt from episodes; correction supersedes inference; model repetition does not become evidence
Echo	Read-only status/verify works; each action is explicit, logged, reversible, and Human Gate controlled
Audio Relay	Record, transcribe, synthesize, play, hotkeys, and failure paths pass independently and end-to-end
Observation Deck	Read-only capture and reconstruction measured against known ground truth; no write authority
Kognitiv Akademia	Current offer and contact assets verified; one real outreach/test cycle completed and recorded
Higher Research	Pre-registered comparison protocol and preserved raw runs
Kirtouma	One accepted definition and boundary before implementation
Archive Recovery	Fixture-complete parser; no truncation; PDFs/registry/manifest tied to source hashes
OS Commissioning	Reproducible inventory, rollback, verified bootstrap, and post-reboot validation

55. Release contents

Each software release should contain only what is needed:

- source;
- exact configuration template without secrets;
- installer if proven;
- verifier;
- rollback or uninstall path;
- tests and fixtures;
- README with boundaries and known limitations;
- artifact manifest and SHA-256 file;
- version and predecessor reference;
- verification record tied to exact hashes.

Do not include stale caches, live ledgers, personal data, model blobs, or environment-specific secrets.

56. Continuation record

At every stopping point, record:

- current goal;
- exact project and module;
- canonical file paths and hashes;
- last successful test;
- first failure, if any;
- changes made;
- unresolved conflicts;
- current Human Gate state;
- exact next read-only action.

Part XVI — Immediate recovery checklist

57. First session

- [] Confirm the target machine and current laboratory root.
- [] Capture read-only system and project inventory.
- [] Hash all files before reorganizing anything.
- [] Identify Git-backed projects and preserve dirty state.
- [] Locate the historical Kaiku candidates and reported hashes.
- [] Locate Module 1 provenance artifacts.
- [] Locate Echo PowerShell ancestor artifacts.
- [] Inspect both Audio Relay install roots.
- [] Locate current Kognitiv, Higher Research, Kirtouma, university, and archive-recovery materials.
- [] Create the artifact conflict register.
- [] Do not move or delete files.

58. Second session

- [] Select the exact Provenance Core candidate using evidence.
- [] Read provenance.py and tests in full.
- [] Run isolated baseline tests.
- [] Verify creation-time permissions and recovery behavior.
- [] Repair one observed defect at most.
- [] Re-test and preserve evidence.
- [] Human Gate on temporary freeze.

59. Following sessions

Proceed in this order:

1. context.py;
2. model.py;
3. rules.py and interaction profile;
4. core.py;
5. kaiku.py;
6. blank user learning;
7. Echo;
8. Audio Relay;
9. bounded integration;
10. Brett acceptance package;
11. governed research experiment;
12. remaining projects according to current priority.

Appendix A — Source assessment

The supplied source archive was inspected as a 6,290-line, 180,224-byte Markdown file with SHA-256: 21387100f8fb1594f0ecce32082bb81fec0d2cb26041c417e6d0b621853572e8

Its structure is composite rather than canonical. It includes:

- a lean V6 architecture summary;
- teaching and Brett interaction discussions;
- a detailed causal narrative of Kaiku execution;
- repeated Module 1 verification reports;
- repeated GUI package reports;
- terminal ancestor artifact lists;
- working-history prompts;
- a 100-item Copilot/context dump;
- an operating addendum numbered 101-118;
- unrelated conversational material interleaved with project material.

This manual therefore performs controlled reconstruction, not transcription. It retains material conflicts and avoids treating repeated assistant prose as independent evidence.

Appendix B — Known conflicts and unresolved decisions

1. Canonical Kaiku build: lean V6 candidate versus later Personal 0.2/guided variants.
2. Model: Qwen3:4B versus Mistral-family preference and one reported mistral:latest default.
3. Audio Relay root: ~/.local/share/kaiku/audio-relay versus ~/.local/share/audio-relay.
4. Exact project numbering: a numbered laboratory structure is desired, but the complete accepted mapping is not present in the supplied archive.
5. Kirtouma definition: two incompatible responsibilities.

6. Prompt reconstruction: context hashes may not be sufficient without preserved exact rule/profile versions.
7. Streaming durability: desired, but interrupted-stream semantics are not yet specified.
8. Locked rules representation: historical 33-rule source versus approximately 19 normalized runtime rules; equivalence must be tested.
9. Brett package readiness: GUI/fake-model tests were historically reported, but real local-model and end-user acceptance were not established in the supplied evidence.
10. OS and installation state: current conversation includes recent machine facts, but this document has not inspected the user's machine.

Appendix C — Supersession rules

- Echo as the whole product name is superseded by Kaiku; Echo remains the terminal/control surface.
- Windows PowerShell installers are historical ancestors on Linux unless deliberately ported.
- Graph-heavy and cognitive-metaphor V6 candidates are not part of the lean baseline unless an observed requirement reintroduces them.
- Arbitrary probability-of-truth fields remain prohibited until their semantics and calibration are established.
- Global intelligence/personality labels are superseded by local, domain-specific, evidence-linked adaptation.

Appendix D — Final rebuild invariant

At any point, another person should be able to answer, from preserved evidence:

- What physically exists?
- Which exact bytes are canonical?
- What was observed versus inferred?
- What changed and why?
- What test was run against which artifact?
- What failed?
- What remains reversible?
- What requires human authorization?
- What is the single next action?

If the record cannot answer those questions, stop expanding the system and repair the record first.